

Codi: 0487 Entorns de desenrotllament 100h

```
File Edit Search Run Compile Debug Options Window Help
DEMONS\WALLS.PAS 1
DEMONS\BOUNDS.PAS 2
DEMONS\BRICKS.PAS 3
DEMONS\COUNT.PAS 4
DEMONS\BREAKOUT.PAS 5-[[↑]]

{ Turbo Breakout }
{ Copyright (c) 1989,90 by Borland International, Inc. }

program Breakout;
{ Turbo Pascal 6.0 object-oriented example.

This is a version of the classic arcade game, Breakout.

SCREEN.PAS
COUNT.PAS
BRICKS.PAS
BOUNDS.PAS
WALLS.PAS
BREAKOUT.PAS

To build an executable file, compile from the command line with:

1:1
```

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Índex

Tema 1: Desenvolupament de programari

Tema 2. Instal·lació i ús d'entorns de desenvolupament (IDE)

Tema 3. Disseny i realització de proves (Testing)

Tema 4. Optimització i documentació

Tema 5. Eines de Col·laboració, Documentació i Integració de Programari

Tema 6: Disseny i elaboració de diagrames de classes

Tema 7. Diagrames de Comportament en UML

Tema 1: Desenvolupament de programari



1. Concepte de programa informàtic
2. Tipus de codi i tecnologies de virtualització
3. Tipus de llenguatges i paradigmes
4. Característiques dels llenguatges més estesos
5. Fases del desenvolupament d'una aplicació (Cicle de Vida)
6. Obtenció del codi executable: Eines
7. Metodologies Àgils

1. Concepte de programa informàtic

Un programa informàtic és una seqüència lògica i estructurada d'instruccions, codificada mitjançant un llenguatge de programació, que un processador (CPU) interpreta i executa per a transformar unes dades d'entrada (input) en uns resultats útils (output).

Perquè aquest conjunt d'instruccions funcione, requereix un procés de compilació o interpretació, que tradueix el codi font (humà) a codi màquina (binari: 0s i 1s), l'únic que el maquinari de l'ordinador pot entendre.

A nivell arquitectònic, un programa no actua sol. Interacció amb el Sistema Operatiu (SO). Demana permís al SO per utilitzar la memòria RAM, emmagatzemar fitxers al disc dur o utilitzar la targeta gràfica.

També hem de parlar de component estàtic vs. Dinàmic. Quan el programa està desat al disc dur és un fitxer executable (estàtic). Per contra, quan l'usuari el prem i es carrega a la memòria RAM per funcionar, es converteix en un procés (dinàmic).

Comparativa: Programa Informàtic vs. App (Aplicació)

El terme "App" és una abreviatura d'application (aplicació). Tot i que conceptualment totes les apps són programes informàtics, a la pràctica hi ha diferències clau en l'ús actual d'ambdós termes:

- Programa Informàtic (Concepte tradicional)

- Plataforma principal: Dissenyat històricament per a ordinadors de sobretaula, servidors o grans sistemes (Mainframes).
- Complexitat: Acostumen a ser programes robustos, multifuncionals i pesants (ex: sistemes de gestió empresarial ERP, programes de disseny en 3D, bases de dades).
- Instal·lació: Es distribuïen en suports físics (CD/DVD) o executables descarregables (.exe, .dmg) amb assistents d'instal·lació complexos.
- Interfície: Interfícies d'usuari complexes (menús plens d'opcions) o, fins i tot, línies de comandes de text (CLI).

- App / Aplicació (Concepte modern)

- Plataforma principal: Dissenyat específicament per a dispositius mòbils (smartphones, tauletes), wearables (rellotges intel·ligents) o navegadors web (Web Apps).
- Complexitat: Solen ser lleugeres i enfocades. En lloc de fer-ho tot, estan optimitzades per a tasques molt concretes (ex: l'app del banc, l'app del temps, una xarxa social).
- Instal·lació: Es distribueixen i s'actualitzen de forma centralitzada a través de botigues oficials (App Store, Google Play), sovint amb un sol clic.

- **Interfície:** Dissenyada sota la filosofia Mobile First, optimitzada per a pantalles tàctils, gestos (lliscar, prémer) i amb un enfocament radical en l'experiència d'usuari (UX).
- **Integració de maquinari:** Aprofiten constantment sensors nadius del dispositiu com el GPS, la càmera, l'acceleròmetre o les notificacions push.

2. Tipus de codi i tecnologies de virtualització

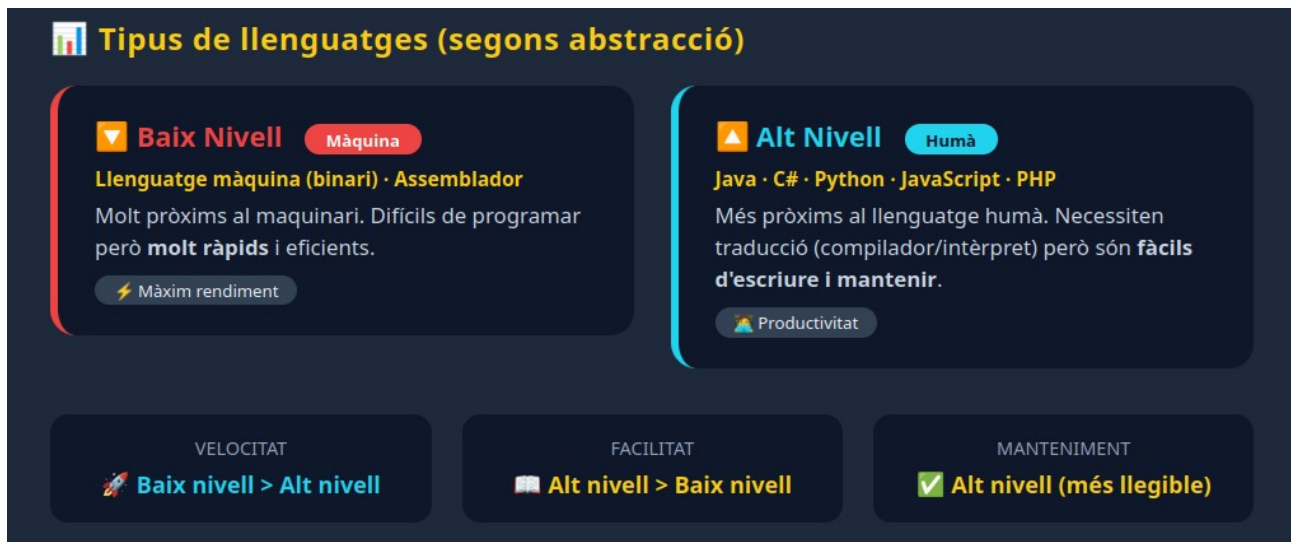
- **Codi font:** Text escrit per programadors en un llenguatge d'alt nivell (entenedor per a humans, ex: Java, Python).
- **Codi objecte:** Codi resultant de la traducció del codi font, normalment en llenguatge màquina o binari, però que encara no està llest per a executar-se perquè li manquen dependències.
- **Codi executable:** Fitxer final que el sistema operatiu pot carregar en memòria i executar directament.
- **Tecnologies de virtualització:** Sistemes que permeten crear una capa d'abstracció sobre el maquinari real.
- **Màquines Virtuals (VM):** Virtualitzen un ordinador complet (inclou sistema operatiu propi).
- **Contenidors (ex: Docker):** Virtualitzen només l'aplicació i les seues dependències compartint el nucli del sistema operatiu amfitrió, sent molt més lleugers.

3. Tipus de llenguatges i paradigmes

Tipus de llenguatges (segons abstracció)

Baix nivell: Llenguatge màquina (binari) i Assemblador. Molt pròxims al maquinari, difícils de programar però molt ràpids.

Alt nivell: Java, C#, Python. Més pròxims al llenguatge humà, necessiten traducció però són fàbrics d'escriure i mantindre.



Paradigmes de programació (estil de programació)

- **Imperatiu:** Es diu a l'ordinador *com* fer les coses pas a pas (ex: C).
- **Orientat a Objectes (POO):** Basat en "classes" i "objectes" que interactuen entre si (ex: Java, C++).
- **Declaratiu / Funcional:** Es definix *què* es vol aconseguir en l'aplicació més que el *com* (ex: SQL, Haskell, html).



4. Característiques dels llenguatges més estesos

- **Java / C#:** Llenguatges orientats a objectes, fortament tipats, multiplataforma (ús de màquina virtual) i molt usats en entorns empresarials.
- **Python:** Llenguatge interpretat, sintaxi molt neta i senzilla, tipat dinàmic. Líder en Intel·ligència Artificial i ciència de dades.
- **JavaScript:** El rei del desenvolupament web (frontend). Executat pel navegador, tot i que ara també es fa servir en servidor (Node.js).
- **C / C++:** Llenguatges de propòsit general de gran potència. C++ afig objectes a C. Control absolut del maquinari i la memòria (jocs, sistemes operatius).

5. Fases del desenvolupament d'una aplicació (Cicle de Vida)



1. **Anàlisi:** Definició de requisits. Saber exactament *què* necessita el client.
2. **Disseny:** Planificació tècnica de l'arquitectura, bases de dades i interfície (UML).
3. **Codificació:** Fase de programació pura en el llenguatge escollit.
4. **Proves (Testing):** Execució de l'aplicació per a detectar errors i verificar el funcionament correcte.
5. **Documentació:** Redacció de manuals tècnics i d'usuari per a futurs canvis o consultes.
6. **Explotació:** Instal·lació de l'aplicació en l'entorn de producció real.
7. **Manteniment:** Correcció d'errors futurs, adaptació a nous entorns i millores.

El cicle que hem descrit és el model tradicional en Cascada (Waterfall). Actualment, el sector tecnològic utilitza alternatives més modernes i flexibles basades en la mentalitat Agile (Àgil) i les pràctiques DevOps.

Aquestes metodologies no eliminen les teves 7 fases, sinó que les executen en cicles molt curts i repetitius (iteracions) en lloc d'una sola vegada al final del projecte.

Les alternatives més utilitzades:

5.1 Metodologies Àgils (Agile).

En lloc de planificar tot el projecte d'una vegada, es divideix el desenvolupament en blocs de temps (generalment de 2 a 4 setmanes) on es repeteixen les 7 fases anteriors a xicoteta escala.

Scrum: Es treballa en "Sprints". Al final de cada Sprint es lliura una part del programari totalment funcional al client perquè el prove.

Kanban: Es basa en la visualització del flux de treball mitjançant targetes en un tauler (Per fer, En procés, Provant, Fet). Evita la sobrecàrrega de feina i optimitza el lliurament continu.

5.2 Cultura DevOps (Development + Operations)

És l'evolució més moderna. Fusiona l'equip de desenvolupament (codificació, proves) amb el d'operacions (explotació, manteniment) perquè treballen junts de forma automatitzada.

CI/CD (Integració Contínua / Desplegament Continu): Cada vegada que un programador escriu codi, uns sistemes automàtics fan les proves (Fase 4) i el pugen directament a producció (Fase 6).

Feedback instantani: El manteniment i l'anàlisi de requisits es fusionen gràcies a la monitorització en temps real de com fan servir l'aplicació els usuaris.

5.3 Disseny de Producte: Lean Startup i MVP

MVP (Producte Mínim Viable): Es llança una versió molt bàsica de l'aplicació amb les 3 o 4 funcions imprescindibles.

Lean Startup (Validació): El mercat real dona el feedback per a decidir quines noves funcionalitats cal dissenyar i programar a continuació, reduint el risc de fracàs econòmic.

6. Obtenció del codi executable: Eines

Per a passar de **Codi Font** a **Codi Executable** s'utilitzen dues vies principals d'eines:

- **Compilador:** Tradueix tot el codi font d'un sol colp i genera un fitxer executable independent (ex: C, C++).
- **Intèrpret:** Tradueix i executa el codi línia per línia en temps real (ex: Python, JavaScript).

- **Model Híbrid (ex: Java):** El codi font es compila a un codi intermedi (Bytecode) que després l'interpret de la Màquina Virtual (JVM) executa en qualsevol sistema.
- **Enllaçador (Linker):** Eina que unix els diferents mòduls de codi objecte i llibreries externes per a crear el fitxer executable final.

7. Metodologies Àgils

En el punt 5 ja n'hem parlat. Són models de treball basats en el desenvolupament iteratiu i incremental, enfocats a l'adaptació ràpida als canvis del client.

- **Característiques:** Lliuraments continus de programari funcional, prioritat del factor humà i col·laboració activa amb el client davant de contractes rígids.
- **Tècniques/Marcs més usats:**
- **Scrum:** El projecte es dividix en blocs de temps fixos (de 2 a 4 setmanes) anomenats *Sprints*. Inclou reunions diàries (*Daily Scrum*) per a coordinar l'equip.

Metodologia Scrum

Marc de treball àgil per al desenvolupament de productes complexos

Rols

Product Owner

Defineix les funcionalitats (Product Backlog). Prioritza per valor de negoci. És la veu del client.

Scrum Master

Facilita el procés, elimina impediments, assegura que l'equip segueixi les regles de Scrum.

Equip de Desenvolupament

Multi-disciplinari (3-9 persones). S'auto-organitza per lliurar els increments de producte.

Esdeveniments (Sprints)

Sprint

Cicle de treball (1-4 setmanes). Es produeix un increment de producte "llest per a producció".

Sprint Planning

Reunió inicial per decidir què es farà al Sprint. Es crea el Sprint Backlog.

Daily Scrum

Reunió diària de 15 minuts per sincronitzar i planificar el dia.

Sprint Review

Demostració del treball fet als stakeholders per obtenir feedback.

Sprint Retrospective

Reunió de l'equip per millorar el procés intern (què fer millor).

Artefactes

Product Backlog

Llista prioritzada de totes les funcionalitats desitjades. La gestiona el Product Owner.

Sprint Backlog

Conjunt d'items del Product Backlog seleccionats per al Sprint actual + el pla per lliurar-los.

Increment

Resultat del Sprint: versió millorada del producte que compleix la "Definició de Fet".

Cicle Scrum

Product Backlog → Sprint Planning → Sprint (2-4 setmanes) → Increment → Review + Retro
→  (torna a començar)

- **Kanban:** Ús d'un tauler visual (Columnes: *Per fer*, *En procés*, *Fet*) per a gestionar el flux de treball i evitar el col·lapse de tasques en paral·lel.

--ooOOoo--

Tema 2. Instal·lació i ús d'entorns de desenvolupament (IDE)



1. Funcions d'un entorn de desenvolupament
2. Instal·lació d'un IDE i llistat d'IDEs més populars
3. Ús bàsic d'un entorn de desenvolupament
4. Personalització de l'entorn de desenvolupament
5. Edició de programes
6. Generació d'executables en distints entorns
7. Eines i automatització

1. Funcions d'un entorn de desenvolupament

Funcions d'un IDE Entorn de Desenvolupament

Un **IDE** (Integrated Development Environment) agrupa en una única interfície totes les eines necessàries per a programar **sense haver d'usar aplicacions separades**.

- Editor de codi intel·ligent**
IntelliSense
Ressaltat de sintaxi, autocompletat i detecció d'errors en **temps real** mentre escrius.
⚡ **Menys errors, més velocitat**
- Compilador / Intèrpret integrat**
Execució directa
Permet executar el codi amb **un sol clic**, sense sortir de l'entorn.
🚀 **Prova immediata**
- Depurador (Debugger)**
Debugging
Permet **pausar l'execució** i revisar les variables **línia per línia** per a buscar errors.
🔍 **Troba errors ràpidament**
- Gestor de projectes**
Organització
Organitza els **fitxers, carpetes i dependències** de l'aplicació de manera estructurada.
📁 **Codi net i ordenat**

📌 Exemples d'IDE populars: Visual Studio, VS Code, IntelliJ IDEA, Eclipse, PyCharm

Un IDE (Integrated Development Environment) agrupa en una única interfície totes les eines necessàries per a programar sense haver d'usar aplicacions separades:

- Editor de codi intel·ligent: Ressaltat de sintaxi, autocompletat (IntelliSense) i detecció d'errors en temps real.
- Compilador / Intèrpret integrat: Permet executar el codi amb un sol clic.
- Depurador (Debugger): Permet pausar l'execució i revisar les variables línia per línia per a buscar errors.
- Gestor de projectes: Organitza els fitxers, carpetes i dependències de l'aplicació.

2. Instal·lació d'un IDE i llistat d'IDEs més populars

El procés general compta amb els següents passos de configuració:

- Instal·lació de l'entorn d'execució / SDK: Abans de l'IDE, cal el motor del llenguatge (ex: el JDK per a Java o el intèrpret per a Python).
- Descarrega de l'IDE: Elecció segons el cicle o perfil (ex: Eclipse o IntelliJ IDEA per a Java; VS Code per a Web).
- Configuració de rutes: Indicar a l'IDE on es troba el compilador o SDK instal·lat al sistema de l'ordinador (configurar la variable d'entorn PATH si cal).

Actualment, els IDEs i editors de codi més populars del món del desenvolupament, classificats segons el seu ús principal, són els següents:

1. L'estàndard global (Multiproòsit) Visual Studio Code (VS Code): Desenvolupat per Microsoft, és l'editor de codi més utilitzat del món. Tècnicament és un editor lleuger, però mitjançant la seua biblioteca d'extensions es transforma en un IDE complet per a qualsevol llenguatge (JavaScript, Python, C++, Go, etc.). Destaca per la seva velocitat, flexibilitat i integració amb Git.

2. Desenvolupament Web i Ecosistema JetBrains: JetBrains crea una suite d'IDEs de pagament (molt utilitzats a nivell professional) especialitzats per a cada tecnologia:

- WebStorm: L'IDE de referència per a desenvolupament web avançat (JavaScript, TypeScript, React, Angular, Vue, Node.js).
- PyCharm: El rei indiscutible per a la programació en Python, ciència de dades i Machine Learning.
- IntelliJ IDEA: L'estàndard de la indústria per al desenvolupament en Java i Kotlin en entorns empresarials.

3. Desenvolupament d'Apps Mòbils:

- Android Studio: L'IDE oficial de Google per crear aplicacions d'Android. Està basat en la tecnologia de JetBrains i està optimitzat per a Kotlin i Java. Inclou emuladors molt potents de telèfons i dispositius de tota mena.
- Xcode: L'IDE exclusiu i oficial d'Apple per programar aplicacions per a iOS, macOS, watchOS i tvOS utilitzant el llenguatge Swift. Només es pot executar en ordinadors Mac.

4. Desenvolupament de Programari Empresarial i Videojocs: Visual Studio, és el germà gran del VS Code, és un IDE pesant i ultra robust de Microsoft. És l'eina estàndard per a aplicacions d'escriptori, serveis en el núvol d'Azure i videojocs (utilitzant el motor Unity), principalment programant en C# i C++.

5. Ciència de Dades i Investigació: Jupyter Notebook / JupyterLab: Un entorn web interactiu molt popular entre científics i analistes de dades. Permet barrejar en un sol document codi executable (majoritàriament Python o R), text en format Markdown, equacions i gràfics integrats.

3. Ús bàsic d'un entorn de desenvolupament

3.1 Ampliació del Flux de Treball Elemental

El cicle de treball operatiu dins d'un IDE va més enllà de la creació física del projecte; engloba el procés de construcció, correcció i gestió del codi:

- Creació del projecte: Selecció del tipus d'aplicació (consola, web, mòbil), versió del llenguatge (SDK/Runtime) i generació automàtica de l'estructura de carpetes i fitxers de configuració basats en plantilles (boilerplates).
- Escripció i assistència (Edició): Redacció del codi font aprofitant les ajudes automàtiques de l'IDE com l'autocompletat intel·ligent (IntelliSense o IA), el ressaltat de sintaxi per colors i la detecció d'errors en temps real (linters).
- Compilació / Construcció (Build): Procés on l'IDE invoca els traductors del llenguatge per transformar el codi font en un fitxer executable o paquet interpretable, verificant que no hi hagi errors de sintaxi.
- Depuració (Debugging): Execució controlada del programa pas a pas. Permet pausar l'aplicació en punts concrets (breakpoints) per inspeccionar el valor de les variables en memòria i localitzar errors lògics.
- Gestió de versions: Sincronització del treball de programació amb repositoris de codi (com Git) directament des de l'IDE per desar l'historial de canvis o treballar en equip.

3.2. Ampliació de l'Estructura de la Interfície

Els IDEs moderns organitzen el seu espai visual en panells especialitzats per optimitzar la productivitat del programador:

- Explorador de projectes: Barra lateral organitzada en forma d'arbre jeràrquic que mostra tots els fitxers de codi, dependències, llibreries externes, imatges i configuracions del projecte.
- Àrea d'edició: L'espai central i principal de la interfície. Treballa amb pestanyes múltiples per tenir diversos fitxers oberts simultàniament, inclou numeració de línies i mapes del codi per a una navegació ràpida.
- Consola / Terminal integrat: Finestra inferior que actua com a intèrpret de comandes del sistema operatiu per a llançar scripts, instal·lar paquets de programari o veure l'eixida de text de l'aplicació en execució.
- Panell de problemes i errors (Output / Diagnostics): Pestanya adjacent a la consola que llista de manera endreçada tots els errors, avisos (warnings) i suggeriments de millora del codi, indicant la línia exacta on es troben.
- Barra d'eines de control d'execució: Zona superior amb botons d'accés ràpid gràfics (icones de "Play", "Stop", "Pause") per a compilar, executar el programa de cop o iniciar el mode depuració pas a pas.

Taula de dreceres visuals per a l'estudiant / usuari

Element de la interfície	Funció principal	Què hi veu l'usuari?
--------------------------	------------------	----------------------

Explorador	Organització	Carpetes, fitxers .py, .java, .js, etc.
Editor	Creació	Text amb colors, paraules clau ressaltades.
Consola	Resultat	Text blanc sobre fons negre amb el resultat del programa.
Panell d'errors	Correcció	Llista de missatges en roig (errors) o groc (avisos).

4. Personalització de l'entorn de desenvolupament

Adaptar l'entorn optimitza el rendiment del programador i evita la fatiga visual:

- Temes i aspecte: Canvi de la interfície a mode fosc (Dark mode) o canvi de la tipografia (ex: Fira Code per a lligadures de programació).
- Estils de codificació: Configuració de regles de format (mida de la tabulació, espais, col·locació de claus { }) per a complir els estàndards de l'equip.
- Mòduls i extensions (Plugins): Afegir noves funcionalitats a l'IDE (ex: extensions de Git, connectors de bases de dades o assistents d'IA).

5. Edició de programes

Funcions avançades de l'editor que estalvien temps i eviten tasques repetitives:

Refactorització bàsica: Canviar el nom d'una variable o funció en tot el projecte de colp de manera segura (Rename).

Generació automàtica de codi: Creació automàtica d'estructures repetitives com els mètodes getters, setters i constructors en Java.

Snippets: Plantilles de codi curtes que s'autocompleten en escriure una drecera (ex: escriure `sout` i que l'IDE el transformi en `System.out.println();`).

6. Generació d'executables en diferents entorns

Segons la tecnologia, el resultat final del projecte es genera de manera diferent des de l'IDE:

Entorns compilats (C/C++): Genera un fitxer binari natiu segons el sistema operatiu (un .exe a Windows o un binari a Linux).

Entorns virtualitzats (Java): Genera un paquet empaquetat .jar que conté el bytecode lliurable, executable en qualsevol ordinador que tinga instal·lada la màquina virtual de Java.

Entorns web (Frontend): No generen un executable; l'IDE organitza els fitxers .html, .css i .js per a ser enviats directament al servidor web.

7. Eines i automatització

Els projectes reals requereixen automatitzar tasques complexes de construcció, proves i gestió de llibreries externes mitjançant gestors de dependències:

Principals eines de l'ecosistema:

- **Maven / Gradle**: Les eines estàndard en el món Java. Automatitzen la descàrrega de llibreries de tercers, executen les proves unitàries i empaqueten el codi final.
- **npm**: El gestor de paquets per a entorns JavaScript i aplicacions web.

Benefici clau: Eviten haver de buscar i descarregar fitxers de llibreries manualment a internet, automatitzant tot el cicle de construcció del programari.

--ooOOoo--

Tema 3. Disseny i realització de proves (Testing)



1. Planificació de proves
2. Tipus de proves
3. Procediments i casos de prova
4. Pruebas de Código (Tècniques de disseny)
5. Pruebas unitarias y herramientas de automatización
6. Documentació d'incidències
7. Dobles de prova (Test Doubles)

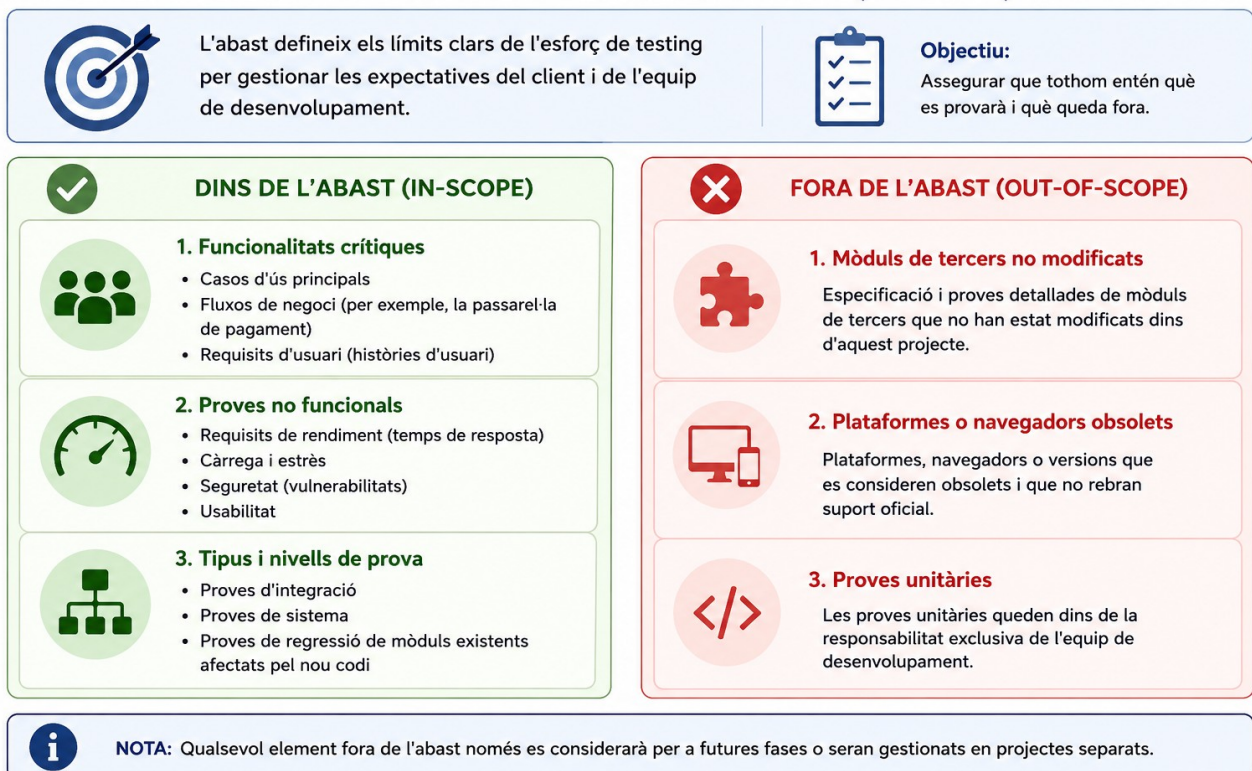
1. Planificació de proves

La planificació de proves és la fase inicial i estratègica del cicle de vida de les proves (STLC - Software Testing Life Cycle). El seu objectiu és alinear els esforços de qualitat amb els objectius de negoci i els terminis del projecte, minimitzant els riscos de desviació en cost i temps.

Tota aquesta estratègia es formalitza en el Pla de Proves (Test Plan), un document viu que s'estructura a partir dels següents eixos fonamentals:

1.1 Abast de les proves (Scope: Què es provarà i què no).

1.1 ABAST DE LES PROVES (SCOPE)



L'abast defineix els límits clars de l'esforç de testing per gestionar les expectatives del client i de l'equip de desenvolupament.

- Dins de l'abast (In-Scope):

- Funcionalitats crítiques: Casos d'ús principals, fluxos de negoci (per exemple, la passarel·la de pagament) i requisits d'usuari (històries d'usuari).
- Proves no funcionals: Requisits de rendiment (temps de resposta), càrrega, estrès, seguretat (vulnerabilitats) i usabilitat.
- Tipus i nivells de prova: Proves d'integració, proves de sistema i proves de regressió de mòduls existents afectats pel nou codi.

- **Fora de l'abast (Out-of-Scope):** Especificació de mòduls de tercers no modificats, plataformes o navegadors obsolets que no rebran suport, o proves unitàries (si aquestes queden dins de la responsabilitat exclusiva de l'equip de desenvolupament).

1.2 Gestió de recursos, entorns i eines (Resources & Infrastructure)

Es detalla la infraestructura logística i humana necessària per poder executar les proves de manera eficient.

- Recursos Humans: Definició de rols i responsabilitats clares. Qui dissenya els casos (QA Engineers), qui els executa (Testers), qui lidera l'estratègia (Test Lead/QA Manager) i qui valida el resultat final (Product Owner / Usuaris clau en proves UAT).
- Entorn de proves (Test Environment): Configuració del programari i maquinari que simula l'entorn de producció. Inclou la disposició de servidors de Staging o QA, la configuració de xarxes i el subministrament de dades de prova (dades fictícies però reals, o dades anonimitzades).
- Eines de Testing: Selecció del programari que s'utilitzarà durant el cicle:
 - Gestió i traçabilitat: Jira, TestRail, Xray.
 - Automatització: Selenium, Cypress, Playwright.
 - Rendiment i API: JMeter, Postman.

1.3 Criteris d'acceptació, èxit i eixida (Criteria & Quality Gates)

Són les mètriques i condicions objectives que determinen la viabilitat de passar el programari a la següent fase o a producció.

Criteris d'entrada (Entry Criteria): Condicions necessàries per començar a provar (ex: codi desplegat correctament a l'entorn de QA, històries d'usuari completament definides i un percentatge de proves unitàries superat).

Criteris d'eixida (Exit Criteria / Success Criteria):

- Cobertura de requisits: El 100% dels casos de prova crítics s'han executat.
- Densitat de defectes: Zero errors de severitat "Bloquejant" o "Alta" oberts. Els errors de severitat baixa o estètica han d'estar documentats i acceptats pel comitè de llançament.
- Taxa d'èxit: Un percentatge fixat (ex: 95%) de tots els casos de prova executats han de tenir estat Pass (Aprovat).

1.4 Anàlisi de Riscos i Contingències (Risk Management)

La planificació ha d'anticipar els problemes potencials que podrien retardar el testing:

Riscos de producte: Mòduls d'alta complexitat amb probabilitat d'errors greus (es planifiquen més proves en aquestes zones mitjançant Risk-Based Testing).

Riscos de projecte: Retards en el lliurament del codi per part de desenvolupament, caigudes de l'entorn de proves o falta de personal. Per a cadascun es defineix una acció de mitigació (ex: reassignació de tasques o automatització de fons).

2. Tipus de proves

2.1 Funcionals (Caixa Negra)

Les proves funcionals basades en la tècnica de Caixa Negra (Black-Box Testing) avaluen què fa el sistema sense preocupar-se per com ho fa internament. El tester no té accés al codi font ni a l'arquitectura de l'aplicació; es comporta com un usuari final, introduint unes dades d'entrada (inputs) i verificant que els resultats obtinguts (outputs) coincideixin amb els requisits especificats.

Tipus de proves funcionals (Caixa Negra)

a) Segons l'objectiu de la validació

Proves de fum (Smoke Testing / Sanity Testing):

Objectiu: Validar de forma ràpida que les funcions més crítiques i bàsiques del sistema funcionen després d'un nou desplegament (ex: comprovar que l'aplicació arrenca i permet fer login).

Utilitat: Actuen com a filtre d'entrada; si aquesta prova falla, es rebutja la versió i no es perd temps fent proves més profundes.

Proves d'Interfície d'Usuari (UI Testing):

Objectiu: Verificar que els elements visuals de la pantalla (botons, camps de text, menús) reaccionen correctament a les accions de l'usuari segons la lògica de negoci de la pantalla.

Proves de Regressió (Regression Testing):

Objectiu: Tornar a executar proves que ja havien passat correctament en el passat per assegurar que els nous canvis de codi, correccions d'errors o actualitzacions no han fet malbé funcionalitats que ja eren estables.

Proves d'Acceptació de l'Usuari (UAT - User Acceptance Testing):

Objectiu: Proves finals realitzades pels usuaris finals o clients en un entorn similar al de producció per confirmar que el sistema compleix amb les seves expectatives de negoci i contractes abans del llançament oficial.

b) Segons el nivell d'integració

Proves del Sistema (System Testing):

Objectiu: Avaluar el programari complet i totalment integrat de punta a punta (end-to-end o E2E) per verificar que compleix amb la totalitat dels requisits funcionals especificats.

Proves d'Integració (Integration Testing):

Objectiu: Provar com interactuen diferents mòduls, components o sistemes externs entre si (ex: verificar que la teva aplicació envia correctament les dades a la passarel·la de pagament de PayPal o Stripe).

Proves d'API (API Testing):

Objectiu: Validar la lògica de negoci a nivell de capa de serveis sense interfície gràfica (provant peticions HTTP GET, POST, PUT, DELETE), verificant que les respostes de dades i els codis d'estat (com el 200 OK) siguin correctes.

c) Tècniques de disseny de Casos de Prova de Caixa Negra

Per no haver de provar infinites combinacions, els testers utilitzen tècniques matemàtiques i lògiques per triar les millors dades d'entrada:

- **Partició d'Equivalència (Equivalence Partitioning):** Es divideixen les dades d'entrada en grups (classes) que el sistema hauria de tractar igual. Es prova només un valor de cada grup.
- **Anàlisi de Valors Límit (Boundary Value Analysis):** Se sap que la majoria d'errors es concentren en els extrems dels rangs permesos. Es proven els valors exactament sobre els límits, just per sobre i just per sota (ex: si un camp accepta edats de 18 a 65 anys, es provaran els valors 17, 18, 19, 64, 65 i 66).
- **Taules de Decisió (Decision Table Testing):** S'utilitzen quan hi ha combinacions complexes de condicions de negoci que generen diferents accions o resultats.
- **Transició d'Estats (State Transition Testing):** S'avalua com el sistema canvia d'un estat a un altre en funció d'accions externes (ex: un caixer automàtic que passa d'estat "Surt de servei" a "Esperant targeta").

2.2 Estructurals (Caixa Blanca):

Es provarà *com* està fet el programa. Es revisa l'estructura lògica del codi font (bucles, condicions, camins).

El proves Estructurals o de Caixa Blanca (White-Box Testing) es basen en l'anàlisi de la implementació interna. El tester té accés total al codi font i dissenya els casos de prova per verificar que l'estructura lògica del programa siga correcta, eficient i no continga camins ocults que puguin fallar.

Què es revisa exactament?

- Cobertura de sentències: Es verifica que cada línia de codi s'execute almenys una vegada.
- Cobertura de branques (condicions): S'assegura que en cada sentència condicional (com un if-else), es proven tant els camins vertaders com els falsos.
- Cobertura de bucles: Es comprova el comportament dels bucles (for, while) quan no s'executen, quan s'executen una vegada, diverses vegades o al límit de la seua capacitat.
- Flux de dades: Es rastreja el cicle de vida de les variables (on es creen, com canvien de valor i on s'utilitzen) per evitar que s'usen variables buides o sense definir.

→ Directori comparatiu d'enfocaments

Per a entendre-ho de manera senzilla, imagina que estem provant un **cotxe**:

Tipus de Prova	Analogia amb un vehicle	En Programació
Caixa Blanca	Obrir el capó per revisar si els engranatges, el motor i els cables estan ben connectats.	Revisar el codi font: comprovar bucles, funcions internes i condicions.
Caixa Negra	Pujar al cotxe, girar la clau i veure si s'encén, accelera i frena correctament.	Provar la interfície: introduir dades en un formulari i veure si funciona.

2.3 De regressió

Proves que s'executen després de modificar el codi (per a corregir un error o afegir una millora) per a assegurar que els canvis no han trencat parts que ja funcionaven.

Aquest concepte és una de les peces més importants de la fase de testing, ja que és el que realment garanteix que un programa pugui evolucionar sense por a fer "dos passos avant i un arrere".

Les proves de regressió no busquen necessàriament nous errors en les funcions que s'acaben de programar, sinó que es dediquen a assegurar que el programari no ha "retrocedit" (regressat) a un estat defectuós en zones que ja havien sigut validades i funcionaven correctament.

Quan s'han d'executar de manera obligatòria?

- Després de corregir un error (Bug fix): S'ha de comprovar que el canvi fet per solucionar l'error "A" no ha provocat col·lateralment l'aparició d'un error "B" en una altra banda del codi.
- Després d'afegir una nova funcionalitat (Feature): Quan el programa creix, el codi nou s'integra amb el vell; cal garantir que no hi haja incompatibilitats.

- Després d'una actualització d'entorn: Si es canvia la versió del llenguatge de programació, de la base de dades o de les llibreries externes utilitzades.

Estratègies per a executar Proves de Regressió

Depenent del temps disponible i de la mida del projecte, es poden aplicar diferents enfocaments:

- Provar-ho tot (Retest All): Es tornen a executar absolutament tots els casos de prova existents al banc de proves. És la forma més segura, però requereix molt de temps i recursos.
- Selecció de proves: S'analitza quin impacte té el canvi introduït i s'executen només les proves de les parts del codi que estan directament o indirectament relacionades amb la modificació.
- Priorització de casos crítics: Es seleccionen primer les proves que validen les funcions vitals de l'aplicació (per exemple, el procés de pagament en una botiga online) per assegurar que el nucli del negoci mai es trenque.

La importància de l'Automatització

Fer proves de regressió de forma manual (un humà fent clics per comprovar que tot continua igual) es torna impossible a mesura que el programa creix. Per això, en el desenvolupament de programari modern:

- Es creen scripts de prova (codi que prova codi).
- S'integren en sistemes de CI/CD (Integració Contínua), de manera que cada vegada que un programador puja un canvi, l'IDE o el servidor executa automàticament centenars de proves de regressió en pocs minuts. Si alguna cosa s'ha trencat, s'avisa a l'equip de seguida.

Exemple pràctic de situació de regressió

Situació inicial: Una aplicació web té un formulari de registre d'usuaris que funciona a la perfecció.

El canvi: El programador modifica el codi del formulari de contacte per a afegir un desplegable de països.

L'error ocult: Per un descuit, el programador altera una llibreria de validació comuna que usen ambdós formularis.

La prova de regressió: El sistema executa la prova del formulari de registre (que no s'hauria d'haver tocat) i aquesta falla. Gràcies a això, l'error es detecta abans que el producte arribi al client.

3. Procediments i Casos de Prova: La documentació del Testing

Per a garantir que les proves siguin repetibles, objectives i que qualsevol membre de l'equip les pugui entendre, el procés de testejat s'ha d'estructurar en dos nivells de documentació complementaris:

3.1 Cas de Prova (Test Case)

Un cas de prova és l'especificació atòmica de què anem a provar. És un document de disseny formal que detalla una condició o escenari de negoci concret per determinar si una característica del programari funciona correctament.

A nivell Professional, un Cas de Prova complet ha d'incloure obligatòriament els següents elements:

- Identificador Únic: Un codi de referència (ex: TC-004).
- Descripció/Objectiu:
- Explicació clara de quina funcionalitat es vol validar.
- Precondicions: L'estat en què ha d'estar el sistema abans de començar (ex: "L'usuari ha de tindre la sessió iniciada").
- Dades d'Entrada (Inputs): Els valors concrets que s'introduiran (ex: Usuari = test@mail.com, Contrasenya = 12345).
- Resultat Esperat: El comportament correcte que hauria de mostrar el programa si no té errors.
- Postcondicions: L'estat en què queda el sistema en acabar (ex: "L'usuari és redirigit al panell de control").

3.2 Procediment de Prova (Test Procedure / Test Script)

Un procediment de prova és l'especificació de com s'executa la prova. És una guia de passos cronològics i detallats que descriu les accions físiques que ha de fer el tester (o l'script d'automatització) per a dur a terme un o més casos de prova de manera sistemàtica.

Què inclou? Inclou instruccions d'ús de la interfície de l'aplicació, camins de navegació (on fer clic, quin menú obrir), com configurar l'entorn de proves i en quin ordre s'han d'anar recollint els resultats.

Relació de jerarquia: Un sol procediment de prova pot agrupar l'execució en cadena de diversos casos de prova relacionats per estalviar temps (per exemple, un sol procediment de "Compra en línia" pot executar el cas de prova d'afegir al carret, el cas de prova d'aplicar descompte i el cas de prova de pagar).

Exemple Pràctic: Plantilla d'un Cas de Prova Real Per a fixar aquests conceptes, vegem com es traduiria un escenari quotidià en documentació de QA per a una pantalla de Login:

Camp de la Plantilla	Contingut del Cas de Prova (TC-012)
Títol	Validació de contrasenya incorrecta al <i>Login</i> .
Precondició	L'usuari ja està registrat en la base de dades. L'aplicació està oberta a la pantalla d'inici de sessió.
Dades d'entrada	Correu: <u>usuari@test.com</u> Contrasenya: <u>1111</u> (Incorrecta).
Procediment (Passos)	1. Escriure el correu en el camp 'Email'. 2. Escriure la contrasenya en el camp 'Password'. 3. Fer clic al botó 'Iniciar Sessió'.
Resultat Esperat	El sistema no permet l'accés. Es mostra un missatge en roig que diu: " <i>Credencials incorrectes, torna-ho a intentar</i> ".
Resultat Real	(Aquest camp s'omple al moment d'executar-la: Passa  / Falla x)

4. Proves de Codi (Tècniques de disseny de Casos de Prova)

Per a dissenyar casos de prova eficients, no es tracta d'introduir dades a l'atzar, sinó d'aplicar mètodes sistemàtics que permeten optimitzar el nombre de proves aconseguint la màxima detecció de defectes possible. Aquestes tècniques es dividixen en dos grans enfocaments:

4.1 Tècniques de Caixa Negra (Enfocades en les Dades d'Entrada)

S'utilitzen quan dissenyem proves a partir dels requisits o especificacions del programa, sense mirar el codi per dins.

a) Classes d'Equivalència (*Equivalence Partitioning*)

Consistix a dividir tot el ventall possible de dades d'entrada en blocs o subgrups homogenis anomenats "classes". La teoria diu que si el programa funciona correctament per a una dada d'una classe, funcionarà igual per a qualsevol altra dada d'eixa mateixa classe. D'aquesta manera, només cal provar un element de cada grup. Tenim:

- **Classes Vàlides:** Grups de dades que el programa ha d'acceptar i processar correctament.
- **Classes Invàlides:** Grups de dades errònies o fora de rang que el programa ha de rebutjar de manera controlada (sense penjar-se).

Exemple: Per a un camp d'edat on s'accepten persones majors d'edat (18 a 100 anys), tenim 3 classes:

1. Menors (Invàlida): Es tria el 12.
2. Majors (Vàlida): Es tria el 45.
3. Edat impossible (Invàlida): Es tria el 120 o un nombre negatiu (-5).

b) Anàlisi de Valors Límit (*Boundary Value Analysis*)

És una tècnica complementària a les classes d'equivalència. L'experiència demostra que **la immensa majoria dels errors de programació es cometten en els extrems dels intervals** (per exemple, escriure un operador $\leq$ en lloc d'un $\leqslant$). Consistix a dissenyar proves just en les fronteres, un punt per damunt i un punt per baix d'aquestes.

Exemple d'aplicació: Si una aplicació accepta un rang d'edat de 18 a 100:

- Límit inferior: Es proven els valors 17 (rebutjat), 18 (acceptat) i 19 (acceptat).
- Límit superior: Es proven els valors 99 (acceptat), 100 (acceptat) i 101 (rebutjat).

c) Endevinar l'Errors (*Error Guessing*)

És una tècnica informal on el tester utilitza la seua experiència, intuïció i l'historial d'errors típics per a "endevinar" on pot fallar el codi.

Exemple: Provar d'introduir lletres en camps numèrics, deixar camps en blanc (valors null), utilitzar caràcters especials (@, *, ç) o intentar forçar una divisió per zero.

4.2 Tècniques de Caixa Blanca (Enfocades en la Cobertura del Codi)

S'utilitzen quan dissenyem proves mirant directament l'estructura lògica del codi font. Ací parlem de **Mètriques de Cobertura** (*Code Coverage*), que mesuren el percentatge de codi que s'ha executat durant els tests.

a) Cobertura de Sentències / Línies (*Statement Coverage*)

- **Què mesura?:** El percentatge de línies de codi que s'han executat almenys una vegada.
- **Objectiu:** Arribar idealment al 100% per garantir que no hi ha "codi mort" (instruccions escrites que mai s'arriben a activar en cap circumstància).

b) Cobertura de Branques o Decisions (*Branch Coverage*)

- **Què mesura?:** El percentatge de camins o alternatives lògiques que s'han avaluat. Cada estructura condicional (com un if / else, un switch o un bucle while) genera bifurcacions.
- **Objectiu:** S'ha de dissenyar una prova perquè la condició del if siga **Vertadera** i una altra prova diferent perquè siga **Falsa**, assegurant que ambdós blocs de codi funcionen. *Nota: Tenir el 100% de cobertura de línies no garantix el 100% de cobertura de branques.*

c) Cobertura de Camins (*Path Coverage*)

- **Què mesura?:** Totes les combinacions possibles de camins que pot seguir el flux del programa des del principi fins al final.
- **Objectiu:** És el nivell més alt i exigent de cobertura. En programes complexos amb molts if niats o bucles, el nombre de camins possibles creix de forma exponencial, per la qual cosa s'utilitzen fórmules matemàtiques (com la *Complexitat Ciclemàtica*) per saber quants camins mínims s'han de provar obligatòriament.

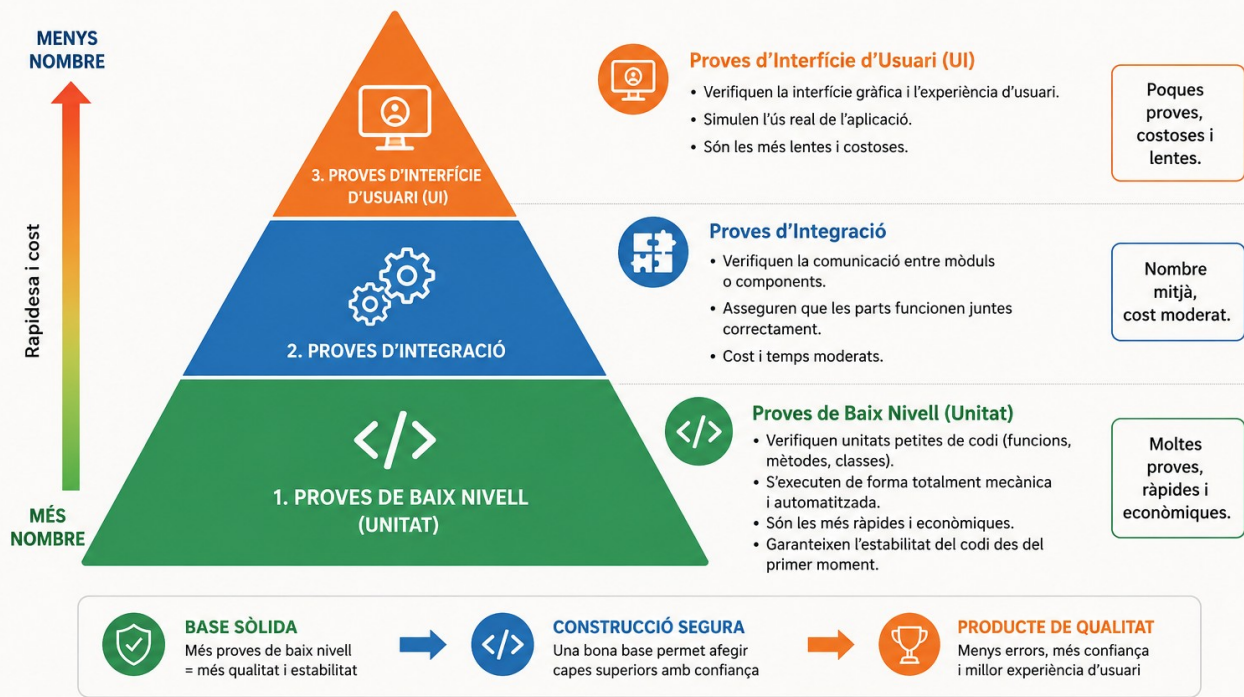
Resum de criteris de disseny:

Tècnica de Disseny	Enfocament principal	Benefici Clau
Classes d'Equivalència	Reduir volum d'entrades	Estalvia temps evitant provar dades redundants.
Valors Límit	Polir les fronteres	Detecta els errors típics de signes (>, <, =).
Cobertura de Branques	Estructura interna del codi	Evita que camins alternatius (<i>else</i>) continguen errors ocults.

5. Proves unitaries i eines d'automatització

LA PIRÀMIDE DE PROVES DEL PROGRAMARI

La base i el component més important són les proves de baix nivell, les quals s'executen de forma totalment mecànica per garantir l'estabilitat del codi des del primer moment.



Dins de la piràmide de proves del programari, la base i el component més important són les proves de baix nivell, les quals s'executen de forma totalment mecànica per garantir l'estabilitat del codi des del primer moment.

5.1 Proves Unitàries (Unit Testing)

Les **proves unitàries** tenen com a objectiu comprovar que la unitat mínima de codi font amb lògica pròpia (generalment una funció, un mètode o una classe) funciona exactament com s'espera.

Perquè una prova siga considerada realment "unitària", ha de complir tres característiques fonamentals:

- **Aïllament total (Independència):** La unitat s'ha de provar de forma completament separada de la resta del sistema. Si el mètode que estem provant necessita connectar-se a una base de dades, a internet o llegir un fitxer del disc dur, eixes dependències externes es substitueixen per objectes simulats artificials (anomenats **Mocks** o **Stubs**). D'aquesta manera, si la prova falla, sabem *de totes totes* que l'error està en la funció i no en la xarxa o la base de dades.
- **Rapidesa:** S'han d'executar en pocs mil·lisegons. Un programador pot tindre milers de proves unitàries al seu projecte i s'han de poder passar totes en qüestió de segons.
- **Repetibilitat:** Han de ser deterministes. Donades les mateixes entrades, una prova unitària ha de retornar **sempre** el mateix resultat, sense importar l'ordre en què s'execute o l'ordinador on es faça.

5.2 Eines d'Automatització i Frameworks de Testing

Executar les proves manualment obrint el programa seria un procés lent i ineficient. Els **frameworks d'automatització** són llibreries especialitzades que s'encarreguen de gestionar, llançar i analitzar el resultat de centenars de casos de prova de colp, generant informes visuals instantanis per al desenvolupador.

El seu funcionament es basa en les **Assercions (Assertions)**: expressions lògiques on es compara el resultat real obtingut per la funció amb el resultat esperat que s'havia planificat. Si l'asserció és vertadera, la prova passa (es mostra en **verd**); si és falsa, la prova falla (es mostra en **roig**).

1. L'estàndard de referència: JUnit (Ecosistema Java)

JUnit és el framework de proves unitàries més utilitzat en el llenguatge **Java**. En les seues versions modernes (com JUnit 5), s'integra directament amb els IDEs (Eclipse, IntelliJ, VS Code) i utilitza **anotacions** (paraules clau que comencen amb **@**) per a definir el comportament de les proves.

Exemple Pràctic: Codi font d'una prova amb JUnit

Per a entendre com es tradueix això a la realitat, vegem com es provaria una classe calculadora molt senzilla en Java:

1. El codi de l'aplicació (Calculadora.java):

```
public class Calculadora {
    public int dividir(int dividend, int divisor) {
        if (divisor == 0) {
            throw new IllegalArgumentException("No es pot dividir per
zero");
        }
        return dividend / divisor;
    }
}
```

2. El codi de la prova automatitzada amb JUnit (CalculadoraTest.java):

java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculadoraTest {

    @Test
    public void testDividirCorrecte() {
        Calculadora calc = new Calculadora();
```

```

        // Executem el mètode i comprovem el resultat (Asserccions)
        int resultatReal = calc.dividir(10, 2);

        assertEquals(5, resultatReal, "La divisió de 10 entre 2 hauria de
ser 5");
    }

    @Test
    public void testDividirPerZeroLlançaExcepció() {
        Calculadora calc = new Calculadora();

        // Verifiquem que el programa reacciona correctament davant d'una
dada invàlida (Valors límit)
        assertThrows(IllegalArgumentException.class, () -> {
            calc.dividir(10, 0);
        });
    }
}

```

Què veu el programador a l'IDE en llançar JUnit?

L'entorn de desenvolupament obrirà una xicoteta pestanya lateral:

- Si tot és correcte, veurà una barra o un cercle **verd** amb el text: Tests passed: 2 of 2.
- Si modifiquem el codi de la calculadora i trenquem alguna cosa, la barra es tornarà **roja**, indicant exactament quin mètode ha fallat, en quina línia i quina era la diferència entre el valor esperat i el real.

6. Documentació i Gestió d'Incidències (*Bug Reporting*)

Quan l'execució d'un cas de prova dona un resultat real diferent de l'esperat, s'identifica un defecte o error (*bug*). Aquest defecte s'ha de registrar immediatament de forma centralitzada en una plataforma de gestió de projectes (com **Jira**, **GitHub Issues**, **GitLab** o **Redmine**) per al seu seguiment.

Un **Informe d'Incidència (*Bug Report*)** professional ha de contindre de manera estructurada els següents elements clau:

- **Identificador Únic i Títol:** Un codi numèric automàtic (ex: BUG-404) acompanyat d'un resum gràfic en format *Acció + Condició + Resultat* (ex: "*L'aplicació es tanca en prémer el botó 'Pagar' si el carret conté més de 5 productes*").
- **Descripció:** Explicació breu de la falla i el context on s'ha produït.
- **Entorn de Proves (*Environment*):** Detall de la configuració tècnica on ha aparegut l'error. Inclou la versió de l'aplicació, el sistema operatiu (ex: Windows 11, iOS 17) i el navegador web o model de dispositiu mòbil utilitzat.

- **Passos per a reproduir-lo (*Steps to Reproduce*):** Una llista numerada i ultra precisa de les accions exactes que ha de fer el programador per a recrear l'error al seu ordinador.
- **Dades d'entrada i Evidències:** Els valors concrets utilitzats i captures de pantalla, enregistraments de vídeo del comportament o fitxers de registre del sistema (*logs*) que demostrin la fallada.
- **Resultat obtingut vs. Resultat esperat:** *Resultat obtingut:* Què fa realment el programa (ex: "Mostra una pantalla en blanc"). *Resultat esperat:* Què hauria de fer segons els requisits (ex: "Hauria de mostrar un missatge de confirmació de compra").

Triage: Distinció entre Gravetat i Prioritat

Una de les tasques més importants a l'hora de documentar un defecte és classificar el seu impacte per a saber quin s'ha de solucionar primer. Són dos conceptes completament diferents:

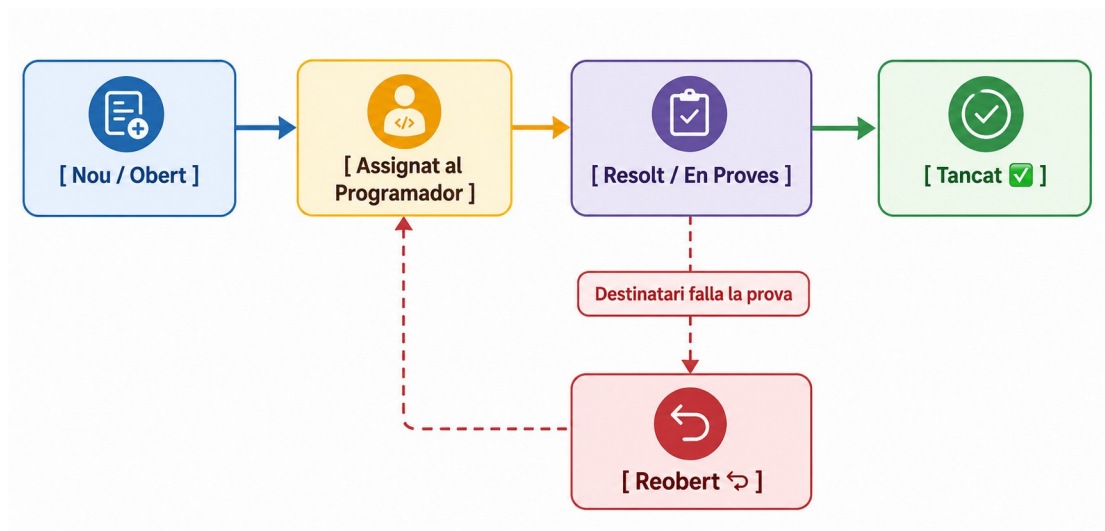
1. **Gravetat (*Severity*):** Mesura l'**impacte tècnic** de l'error en el funcionament del programari. És una mètrica objectiva determinada pel *tester*.
 - *Crítica:* El programa es penja, es perden dades o una funció vital està totalment bloquejada.
 - *Major / Menor:* Funcions secundàries fallen, però hi ha alternatives per continuar usant l'app.
 - *Trivial / Cosmètica:* Un error ortogràfic, un color equivocat o un text mal alineat.
2. **Prioritat (*Priority*):** Mesura l'**impacte de negoci** i determina la urgència temporal amb què el programador ha de corregir-lo. És una decisió de gestió (del *Product Owner* o cap de projecte).
 - *Alta:* S'ha de solucionar immediatament (pot ser que bloquege el llançament al mercat).
 - *Mitjana / Baixa:* Es pot solucionar en els pròxims dies o en futures versions del programa.

L'exemple clàssic de la diferència entre Gravetat i Prioritat:

- Un error on el logotip de l'empresa a la pantalla principal apareix Pixelat: Té una **Gravetat Baixa** (el programa funciona al 100%), però una **Prioritat Alta** (fa molt mala imatge comercial de cara al client).
- Un error on el botó de descàrrega dels "Termes i Condicions de l'any 2012" fa fallar el servidor: Té una **Gravetat Alta** (trenca el sistema), però una **Prioritat Baixa** (ningú fa clic en eixe botó vell).

Exemple de cicle de vida d'un Bug a Jira

Més enllà de la documentació, una incidència passa per diferents **estats** en un tauler de gestió:



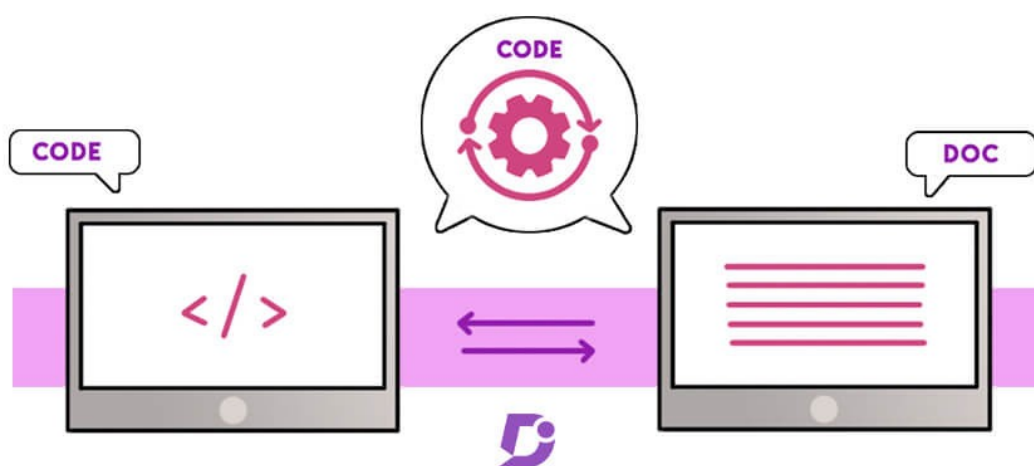
7. Dobles de prova (Test Doubles)

Quan es fa una prova unitària, el mètode a provar pot dependre d'altres elements (com una base de dades o una API externa). Per a aïllar la prova i no dependre d'estructures reals, s'usen **dobles de prova**:

Tipus i característiques:

- **Dummy:** Objectes buits que es passen com a paràmetres només perquè el codi compile, però mai s'utilitzen realment.
- **Stub:** Objectes que retornen respostes fixes preparades per a la prova, sense lògica interna (ex: simular que la base de dades sempre respon true).
- **Mock:** Objectes sofisticats que no sols retornen dades, sinó que verifiquen si el codi ha interactuat amb ells correctament (ex: comprovar si s'ha cridat al mètode enviarCorreu() exactament una vegada).
- **Fake:** Implementacions reals però simplificades que no serveixen per a producció (ex: usar una base de dades en memòria H2 en lloc d'una base de dades MySQL real).

Tema 4. Optimització i documentació



1. Concepte de Refactorització (*Refactoring*)

La refactorització és el procés de modificar el codi font d'un programa per a millorar la seua estructura interna, llegibilitat i mantenibilitat sense alterar en absolut el seu comportament extern.

- **Què significa "sense alterar el comportament extern"?** Significa que, per a l'usuari final, el programa fa exactament el mateix abans i després de la refactorització. Les pantalles funcionen igual, es demanen les mateixes dades d'entrada (*inputs*) i es generen els mateixos resultats (*outputs*).
- **Quina és la diferència amb arreglar un bug?** La refactorització **no corregix errors** ni afegix noves funcionalitats. El seu únic objectiu és fer que el codi siga més net, més fàcil d'entendre per a altres programadors i més flexible davant de futurs canvis.

Per què és necessària? El Deute Tècnic.

A mesura que un programa creix i s'afegixen canvis de pressa, el codi es degrada de manera natural (un fenomen conegut com a *entropia del programari*). Si no es refactoritza, apareix el **Deute Tècnic** (*Technical Debt*): l'acumulació de codi mal dissenyat que fa que, en el futur, afegir una simple millora siga lent, costós i propens a provocar nous errors.

Beneficis clau de refactoritzar:

- **Millora la llegibilitat:** Codi més net equival a menys temps intentant entendre què fa una funció.
- **Reduïx la complexitat:** Elimina codi duplicat o camins lògics innecessàriament complicats.
- **Facilita la detecció de bugs:** En un codi ben estructurat, els errors visuals o lògics salten a la vista molt més ràpid.

Olors de Codi (*Code Smells*) i Tècniques Comunes

Els programadors refactoritzen quan detecten "**olors en el codi**", que són símptomes indicatius d'un mal disseny. A continuació es mostren els exemples més habituals i com es solucionen:

- **Codi Duplicat:** El mateix fragment de codi s'ha copiat i enganxat en tres llocs diferents.
Solució: Es crea una única funció comuna i es crida des d'eixos tres punts (**Extracció de mètode**).
- **Mètodes massa llargs:** Una sola funció té 200 línies de codi i fa moltes tasques alhora.
Solució: Es dividix en 4 o 5 funcions xicotetes, cadascuna especialitzada en una sola cosa (*Principi de Responsabilitat Única*).
- **Noms poc clars:** Variables o funcions amb noms com `x`, `data` o `funcion1()`.
Solució: Es bategen de nou amb noms descriptius (ex: `preuAmblva`, `calcularDescompte()`).

La Regla d'Or de la Refactorització: El Testing

No es pot fer una refactorització segura sense una xarxa de seguretat. **La regla d'or és tindre proves unitàries automatitzades (com JUnit)** creades abans de començar a tocar el codi.

El flux de treball correcte és:

1. Executar les **proves unitàries** en el codi vell (tot ha de complir en **verd** ✓).
2. **Modificar i netejar** el codi per millorar-lo.
3. Tornar a executar les **proves unitàries**. Si continuen en **verd** ✓, la refactorització ha sigut un èxit perquè s'ha demostrat de forma matemàtica que no hem trencat res. Si es tornen **roges** x, hem alterat el comportament extern per error.

Exemple Visual de Refactorització (Codi Java)

Abans (Codi complex i poc clar):

```
// El nom no diu res i fa dues coses: calcula i imprimix
public void p(double b, double i) {
    double r = b + (b * (i / 100));
    System.out.println("El total es: " + r);
}
```

Després de refactoritzar (Noms clars i responsabilitat única):

```
java
// Ara el mètode només calcula, és reutilitzable i fàcil d'entendre
public double calcularPreuTotal(double preuBase, double percentatgeIva) {
    double importIva = preuBase * (percentatgeIva / 100);
    return preuBase + importIva;
}
```

2. Analitzadors de Codi (Anàlisi Estàtica de Programari)

L'Anàlisi Estàtica de Codi (SAST - *Static Application Security Testing*) consisteix a examinar el codi font d'una aplicació abans de ser compilat o executat. És una auditoria automatitzada que llegeix el text del codi línia per línia per verificar que compleix amb els estàndards de qualitat, eficiència i seguretat establerts pel sector.

2.1 Diferència clau en el Testing

Tipus d'Anàlisi	Com funciona?	Quan s'executa?	Què busca?
Estàtica (Static)	Revisa el text del codi font sense executar-lo .	Durant l'escriptura o compilació.	Estil, males pràctiques, seguretat i deute tècnic.
Dinàmica (Dynamic)	Revisa el programa mentre s'està executant (ex: JUnit).	Durant la fase de proves natius.	Error de lògica, rendiment, fuites de memòria.

2.2 Classificació de les Eines d'Anàlisi Estàtica

Les eines es divideixen segons el nivell de profunditat amb què analitzen el codi:

- Linters (Format i Estil)

Són els analitzadors més superficials però imprescindibles. Comproven si el programador segueix les normes estètiques de la comunitat (com l'ús de tabulacions o espais, posició de les claus {}, o el límit de caràcters per línia).

- **Checkstyle (Java):** Garanteix que tot l'equip escriu el codi exactament amb el mateix estil visual.
- **ESLint (JavaScript):** Troba descuits de sintaxi comuns abans que el navegador intente arrancar el codi.

- Analitzadors de Bugs Potencials i Antipatrons

Aquestes eines busquen construccions de codi que, tot i ser sintàcticament correctes, tenen una alta probabilitat de provocar errors en producció (*bugs* semàntics) o vulnerabilitats de seguretat (com injeccions SQL).

- **PMD / SpotBugs (antics FindBugs):** Són connectors (*plugins*) que s'instal·len en l'IDE i analitzen el codi en segon pla. Avisen d'errors típics com:
- Variables declarades que mai s'utilitzen (malbaratament de memòria).
- Comparar textos en Java usant `==` en lloc de `.equals()`.
- Blocs `catch` buits que silencien errors sense registrar-los.

- Plataformes de Qualitat Avançada (Mètriques i Deute Tècnic)

Són sistemes complets (sovint basats en servidors web) on s'analitza tot el projecte de colp per mesurar la salut de l'arquitectura global.

- **SonarQube (L'estàndard de la indústria):** Escaneja el codi i genera un quadre de comandament detallat amb:
- **Deute Tècnic:** Calcula en hores/dies el temps que tardaria un desenvolupador a netejar el codi mal fet.
- **Duplicació de codi:** Indica el percentatge exacte de text copiat i enganxat que s'hauria de refactoritzar.
- **Complexitat Ciclomàtica:** Una mètrica que mesura el nombre de camins independents que té un mètode (quants `if`, `while` o `for` hi ha enllaçats). Si la complexitat és massa alta (ex: superior a 10 o 15), SonarQube avisa que el codi és massa difícil d'entendre i de provar, i obliga a dividir-lo.

- Integració contínua (CI/CD)

En els equips professionals, aquestes eines no es passen de forma manual. S'integren com a "**Portes de Qualitat**" (*Quality Gates*) en els servidors de DevOps. Si un programador puja codi que conté un error de seguretat o augmenta el deute tècnic per damunt dels límits permesos, el sistema rebutja el canvi automàticament i no el deixa publicar.

3. Control de versions

Un sistema de control de versions (VCS) registra els canvis realitzats en els fitxers d'un projecte al llarg del temps, permetent tornar a versions anteriors i treballar en equip sense xafar el codi dels companys.

Estructura de les eines de control de versions: Segons la seua arquitectura, es dividixen en dos tipus

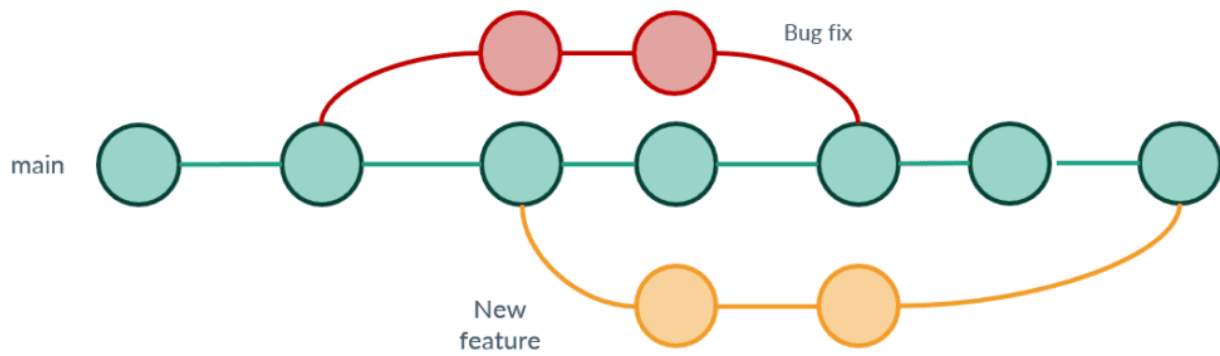
- **Sistemes Centralitzats (ex: Subversion/SVN):** Hi ha un únic servidor central que conté tot l'història. Si el servidor cau, ningú pot treballar ni guardar l'història.
- **Sistemes Distribuïts (ex: Git):** Cada programador té una còpia **completa i exacta** del repositori amb tot l'història al seu propi ordinador. Es treballa en local i després se sincronitzen els canvis amb un servidor remot (com GitHub o GitLab).

En una eina distribuïda com Git, els fitxers passen per tres zones locals abans d'anar al servidor:

1. **Working Directory (Directorio de treball):** On estan els fitxers reals que estàs modificant al teu ordinador.
2. **Staging Area / Index (Àrea de preparació):** Zona intermèdia on prepares i seleccions quins canvis concrets vols incloure a la pròxima foto o versió.
3. **Local Repository (Repositori local):** On es guarden definitivament els canvis en l'història del teu ordinador (mitjançant un *commit*).
4. **Remote Repository (Repositori remot):** El servidor al núvol (GitHub) on puges els canvis (*push*) per a compartir-los amb l'equip.

--ooOOoo--

Tema 5. Eines de Col·laboració, Documentació i Integració de Programari



1. Repositoris remots
2. Documentació i comentaris
3. Integració Contínua (CI - CD)

1. Repositoris remots

Un repositori remot és una còpia del teu projecte allotjada en un servidor a internet o a la xarxa local. Permet la col·laboració entre programadors, actuant com a punt centralitzat on sincronitzar el treball.

Plataformes populars: GitHub, GitLab i Bitbucket.

Operacions bàsiques amb remots:

- **Clone (Clonar):** Descarregar per primera vegada un repositori remot complet al teu ordinador local.
- **Push (Pujar):** Enviar els canvis locals (els teus *commits*) al repositori remot.
- **Pull (Baixar/Actualitzar):** Descarregar els nous canvis que han pujat els teus companys des del remot i fusionar-los (*merge*) automàticament amb el teu codi local.
- **Fetch:** Descarregar la informació del remot per a veure què hi ha de nou, però sense aplicar ni fusionar els canvis encara.

Ús integrat en l'IDE

Els entorns de desenvolupament moderns (IntelliJ, Eclipse, VS Code) eliminen la necessitat d'usar la consola de comandes per a Git. Incorporen interfícies gràfiques on es poden veure les diferències entre versions línia per línia, fer *commits* marcant caselles, gestionar branques de manera visual i resoldre conflictes de codi amb un assistent de fusions.

2. Documentació i comentaris

La documentació de programes és el conjunt de fitxers, textos i comentaris que expliquen com funciona, com s'ha de mantenir i com s'utilitza un programari. Un bon desenvolupament de programari és inseparable d'una documentació clara i actualitzada.

Es divideix principalment en dues grans categories:

Documentació Tècnica (Interna): Està adreçada als **desenvolupadors** i professionals de l'equip informàtic que han de mantenir, modificar o ampliar el codi.

- **Comentaris en el codi font:** Explicacions breus integrades directament en el text del programa per aclarir blocs lògics complexos.
- **Documentació de l'API (Javadoc, Sphinx, Swagger):** Guies generades automàticament que descriuen les funcions, classes, mètodes i els paràmetres de cada component.
- **Arquitectura del sistema:** Diagrames (com UML) i textos que detallen l'estructura global del programari, bases de dades i connexions de xarxa.

- **Manual d'instal·lació i configuració:** Instruccions per muntar l'entorn de desenvolupament o desplegar el programa en un servidor.

Documentació d'Usuari (Externa): Està adreçada als **usuaris finals** o administradors del sistema que utilitzaran l'aplicació sense necessitat de veure'n el codi.

- **Manual d'usuari:** Guia detallada pas a pas sobre com utilitzar totes les funcionalitats del programari.
- **Guia d'inici ràpid (Quick Start):** Un resum breu per començar a fer servir l'aplicació en pocs minuts.
- **Preguntes freqüents (FAQ):** Respostes directes als dubtes i problemes més comuns dels usuaris.
- **Notes de la versió (Release Notes):** Document on s'especifiquen les millores, noves característiques i errors corregits en cada actualització.

Ús de comentaris en el codi:

- *Comentaris de línia o bloc:* S'usen per a explicar per què es fa una acció complexa, mai per a descriure què fa una línia obvia.
- *Comentaris de documentació (JavaDoc):* Comentaris estructurats que es col·loquen just damunt de classes i mètodes utilitzant etiquetes especials com `@param` (per a descriure variables d'entrada) o `@return` (per a descriure el resultat).
- **Generació automàtica:** Mitjançant l'eina javadoc, l'IDE pot llegir aquests comentaris estructurats i generar de manera automàtica una pàgina web (`.html`) amb la documentació tècnica oficial del projecte.
- **Alternatives (Codi Net / Clean Code):** La tendència actual és reduir els comentaris al mínim. L'alternativa és escriure un codi tan net, amb noms de mètodes i variables tan descriptius, que el codi s'explique per si mateix sense necessitat d'afegir línies de text explicatives.

3. Integració Contínua (CI - CD)

La **Integració Contínua (CI)** i el **Desplegament Continu (CD)** són pràctiques de desenvolupament de programari dissenyades per automatitzar el cicle de vida de les aplicacions, reduint els errors en integrar codi i accelerant els lliuraments.

Components Principals: CI vs. CD

Concepte	Què fa?	Objectiu principal
Integració Contínua (CI)	Fusiona el codi dels desenvolupadors en una branca central diverses vegades al dia. Cada fusió activa automàticament la compilació i les proves .	Detectar errors de forma immediata i evitar conflictes de codi.
Lliurament Continu (CD)	Automatitza la preparació de l'aplicació per a la seva publicació. El codi passa automàticament a un entorn de proves o	Deixar el programari sempre a punt per ser llançat a producció amb un sol clic.

	preproducció.	
Desplegament Continu (CD)	Va un pas més enllà: qualsevol canvi que superi les proves es publica automàticament directament als usuaris finals.	Reduir a zero la intervenció humana en el desplegament del codi.

El Flux de Treball Típic (Pipeline)

1. **Codi (Code):** El desenvolupador puja el nou codi al repositori (ex. GitHub, GitLab).
2. **Compilació (Build):** El servidor de CI detecta el canvi i construeix l'aplicació.
3. **Proves (Test):** S'executen les **proves estructurals (caixa blanca)** i unitàries automatitzades. Si falla una prova, el flux s'atura.
4. **Alliberament (Release):** Es genera l'artefacte (un fitxer .jar, una imatge Docker, etc.) i la **documentació de l'API**.
5. **Desplegament (Deploy):** Es puja l'artefacte automàticament als servidors de producció.

Eines més utilitzades

- **Orquestradors:** Jenkins, GitHub Actions, GitLab CI/CD, CircleCI.
- **Contenidors (fonamentals per a CD):** Docker, Kubernetes.

--ooOOoo--

Tema 6: Disseny i elaboració de diagrames de classes



1. Classes i Objectes: Els Blocs Fonamentals
2. Notació d'una Classe en UML
3. Relacions entre Classes
4. Eines de Modelat UML
5. Generació automàtica i Enginyeria Inversa

El disseny de diagrames de classes forma part de UML (Unified Modeling Language) i és una eina clau per estructurar la documentació tècnica d'un programa, connectant directament el disseny conceptual amb el codi font que després es provarà amb tècniques de caixa blanca.

1. Classes i Objectes: Els Blocs Fonamentals

Per entendre un diagrama de classes, primer cal distingir clarament entre aquests dos conceptes de la Programació Orientada a Objectes (POO):

- **La Classe:** És el **plànol, plantilla o motlle**. Defineix de forma abstracta quines característiques i quins comportaments tindran els elements d'aquell tipus. No ocupa espai a la memòria del programa en execució.
- **L'Objecte:** És la **instància real**. És l'element concret creat a partir del plànol de la classe. Ocupa un lloc a la memòria i té dades reals.

Taula comparativa directa

Característica	La Classe (Plànol)	L'Objecte (Instància)
Què és?	La definició teòrica i genèrica.	L'entitat real i tangible.
Quantes n'hi ha?	Només una definició al codi.	Pot haver-hi infinit s objectes d'una mateixa classe.
Exemple analògic	El plànol d'una casa.	La casa de maons on viu algú.
Exemple en codi	La classe <u>Cotxe</u> .	El cotxe roig amb matrícula <u>1234-XYZ</u> .

2. Notació d'una Classe en UML

En un diagrama, una classe es representa com un rectangle dividit en **tres seccions horitzontals**:

1. **Superior:** El nom de la classe (en negreta, centrat, comença en majúscula).
2. **Central:** Els **Atributs** (variables) amb el format: visibilitat nomAtribut : TipusDada.
3. **Inferior:** Els **Mètodes** (funcions/operacions) amb el format: visibilitat nomMetode(paràmetres) : TipusRetorn.

BankAccount
owner : String balance : Dollars = 0
deposit (amount : Dollars) withdrawal (amount : Dollars)

3. Relacions entre Classes

Les relacions definixen com interactuen les classes entre si. Es representen amb diferents tipus de línies i fletxes:

- **Associació:** Relació estructural bàsica entre dues classes (línia contínua simple).
Navegabilitat: Indica la direcció de la relació amb una fletxa >. Si A té una fletxa cap a B, A coneix B, però B no coneix A.
Multiplicitat: Indica quants objectes d'una classe es poden relacionar amb un objecte de l'altra. S'escriu en els extrems de la línia (ex: 1 a un costat, 0..* o * a l'altre).
- **Agregació:** Relació del tipus "té un" o "forma part de", on els objectes tenen cicles de vida independents. Si el tot es destrueix, la part sobreviu (línia contínua amb un **rombe buit** <> en el costat de la classe contenidora). *Exemple: Aula i Ordinadors.*
- **Composició:** Agregació forta on la part no pot existir sense el tot. Si es destrueix l'objecte principal, les parts es destrueixen automàticament (línia contínua amb un **rombe ple** ◆). *Exemple: Llibre i Pàgines.*
- **Herència / Generalització:** Una classe filla hereta els atributs i mètodes d'una classe mare (línia contínua amb una **fletxa triangular buida** ◁ apuntant a la classe mare).
- **Dependència / Ús:** Una classe utilitza a una altra de manera temporal (ex: com a paràmetre d'un mètode) però no la guarda com a atribut (línia **discontínua** amb fletxa simple -->).
- **Realització / Implementació:** Una classe implementa el comportament definit en una interfície (línia **discontínua** amb una **fletxa triangular buida** ◁- apuntant a la interfície).

4. Eines de Modelat UML

Aplicacions utilitzat de forma habitual a les pràctiques per a dibuixar els diagrames:

- **Draw.io / Lucidchart:** Eines web gratuïtes, ràpides i col·laboratives.
- **StarUML / Umbrello:** Programari d'escriptori dedicat exclusivament a UML, molt complet per a l'àmbit acadèmic.
- **PlantUML:** Permet escriure els diagrames en format text amb codi senzill i l'eina els dibuixa automàticament (molt utilitzat integrat com a extensió en VS Code o IntelliJ).

5. Generació automàtica i Enginyeria Inversa

Els IDEs moderns i les eines de modelat avançades connecten el disseny visual amb el codi real:

- **Generació automàtica de codi:** A partir d'un diagrama de classes UML ben definit, l'eina pot crear de manera automàtica l'esquelet dels fitxers de codi (ex: fitxers .java amb les classes, atributs, mètodes i relacions ja estructurades), estalviant faena de picar codi.

- **Enginyeria Inversa:** El procés contrari. L'eina llig un projecte ple de fitxers de codi font i genera automàticament el diagrama de classes UML, ideal per a entendre de colp l'arquitectura d'un projecte ja programat que s'ha d'analitzar.

--ooOOoo--

Tema 7. Diagrames de Comportament en UML

Behavioral Diagrams

Use Case Diagram

Sequence Diagram

Communication Diagram

Interaction Diagram

Activity Diagram

State Chart Diagram

Time Sequence Diagram

1. Tipus de diagrames i Camp d'aplicació
2. Diagrama de Casos d'Ús
3. Diagrama de Seqüència
4. Diagrama de Comunicació
5. Diagrama d'Activitat
- 6 Diagrama d'Estats (Cicle de vida)

Els Diagrames de Comportament en **UML** s'utilitzen per a modelar la dinàmica, el flux de dades i les interaccions del sistema en funcionament (a diferència del diagrama de classes, que és estàtic).

1. Tipus de diagrames i Camp d'aplicació

Mentre els diagrames estructurals mostren *què hi ha* al sistema, els de comportament mostren *què fa* i *com reacciona*.

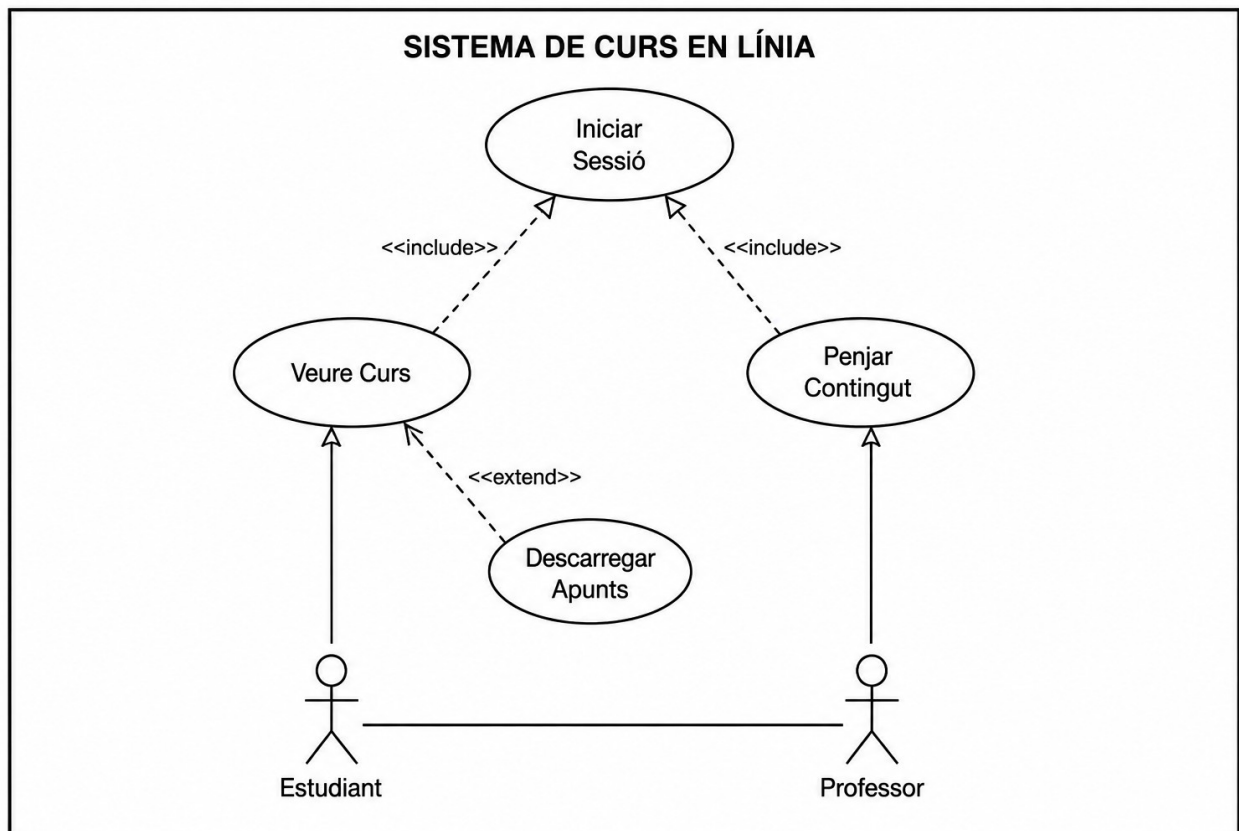
- **Camp d'aplicació:** S'utilitzen en les fases d'**anàlisi** (per a entendre els requisits del client) i de **disseny** (per a planificar la lògica dels algorismes i la interacció entre objectes abans de programar).
- **Classificació principal:** Es dividixen en diagrames de comportament pur (activitats, estats, casos d'ús) i diagrames d'interacció (seqüència, comunicació).

2. Diagrama de Casos d'Ús

S'utilitza en la fase d'anàlisi per a representar de manera visual **les funcionalitats del sistema des del punt de vista de l'usuari**.

- **Actor:** Entitat externa al sistema que interactua amb ell (un usuari, un administrador o fins i tot un altre sistema o base de dades). Es representa amb una figura de pal (*stickman*).
- **Cas d'Ús:** Una operació o tasca concreta que realitza el sistema a petició d'un actor. Es representa amb una **el·lipse**.
- **Relació de comunicació:** Línia contínua que unix l'actor amb el cas d'ús.
- **Escenari:** La descripció detallada del flux de passos que se segueixen per a completar el cas d'ús.
- **Relacions avançades entre Casos d'Ús:**
 - **«include»** (Inclusió): Un cas d'ús obligatòriament en necessita un altre per a funcionar (ex: *Fer un pagament* inclou *Validar targeta*).
 - **«extend»** (Extensió): Un cas d'ús s'executa només de manera opcional en certes condicions (ex: *Fer una compra* pot estendre's a *Aplicar cupó de descompte*).

Exemple:



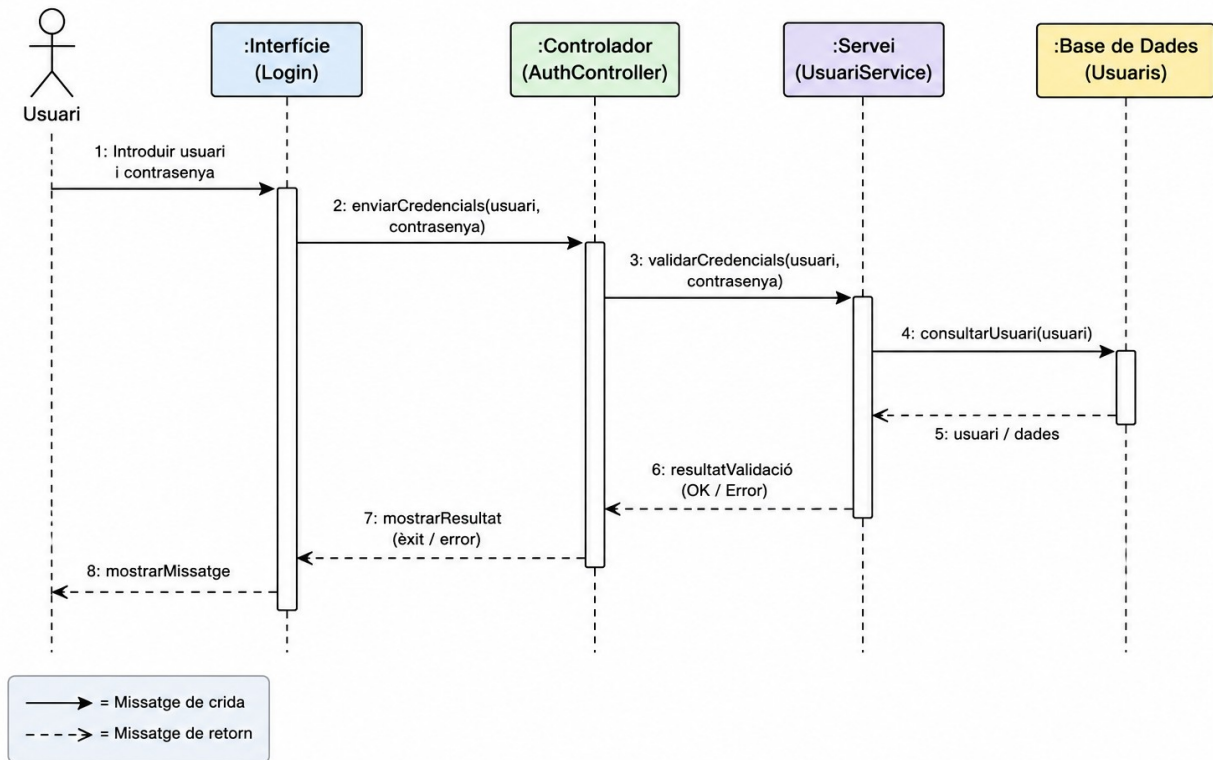
3. Diagrama de Seqüència

Es visualitza la **interacció d'objectes en l'ordre cronològic de temps** en què ocorren els esdeveniments. És crucial per a entendre com es comuniquen les diferents classes del codi.

- **Línia de vida (Lifeline):** Línia vertical discontinua que baixa des del rectangle de l'objecte i representa la seua existència al llarg del temps.
- **Activació (Barra de control):** Rectangle vertical estret sobre la línia de vida que indica que l'objecte està executant una operació en eixe moment.
- **Enviament de missatges:** Línies horitzontals amb fletxes que van d'una línia de vida a una altra:
 - *Missatge síncron:* Línia contínua amb fletxa farcida ($\Rightarrow$). L'emissor espera resposta.
 - *Missatge asíncron:* Línia contínua con fletxa simple ($\rightarrow$). L'emissor continua sense esperar.
 - *Missatge de retorn:* Línia discontinua ($\leftarrow$) per a retornar una dada a l'objecte que ha fet la crida.

Exemple:

Diagrama de Seqüència - Iniciar Sessió



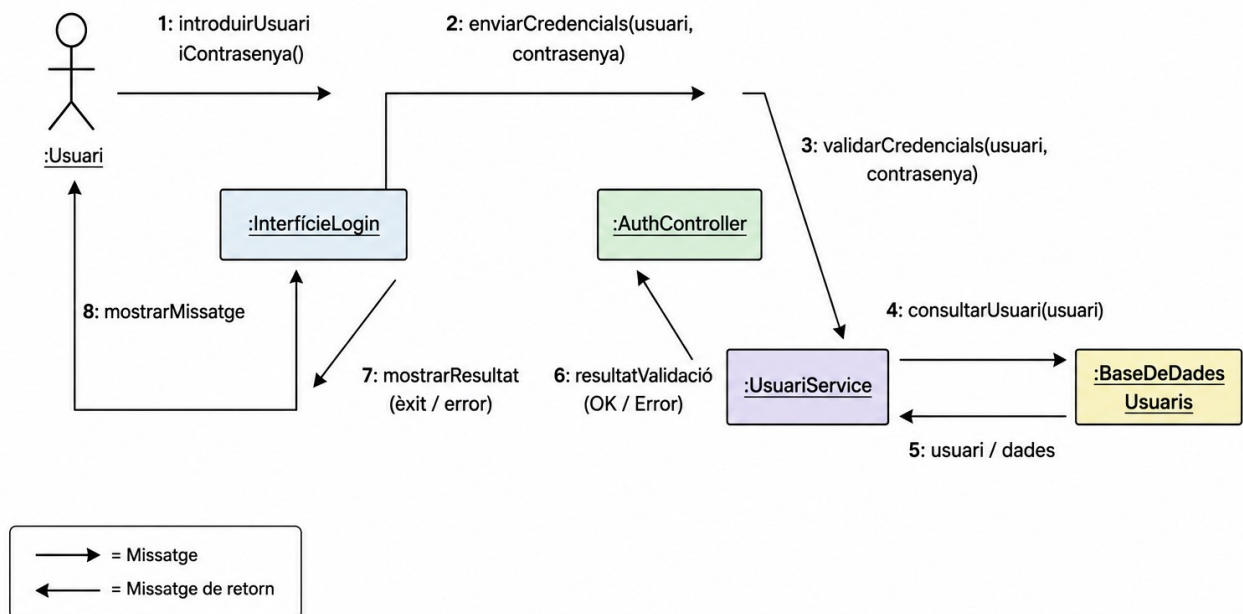
4. Diagrama de Comunicació

Mostra la mateixa informació que el diagrama de seqüència, però **focalitzat en l'estructura d'organització dels objectes** en lloc del temps.

- **Objectes:** Es dibuixen com a rectangles connectats per línies d'associació simples.
- **Missatges:** Es col·loquen fletxes xicotetes paral·leles a les línies d'associació que indiquen la direcció del missatge.
- **Seqüenciació:** Per a saber l'ordre cronològic (ja que no hi ha eix de temps vertical), els missatges **es numeren obligatòriament** (ex: 1. introduirCodi(), 1.1. validarCodi(), 2. mostrarResultat()).

Exemple:

Diagrama de Comunicació - Iniciar Sessió



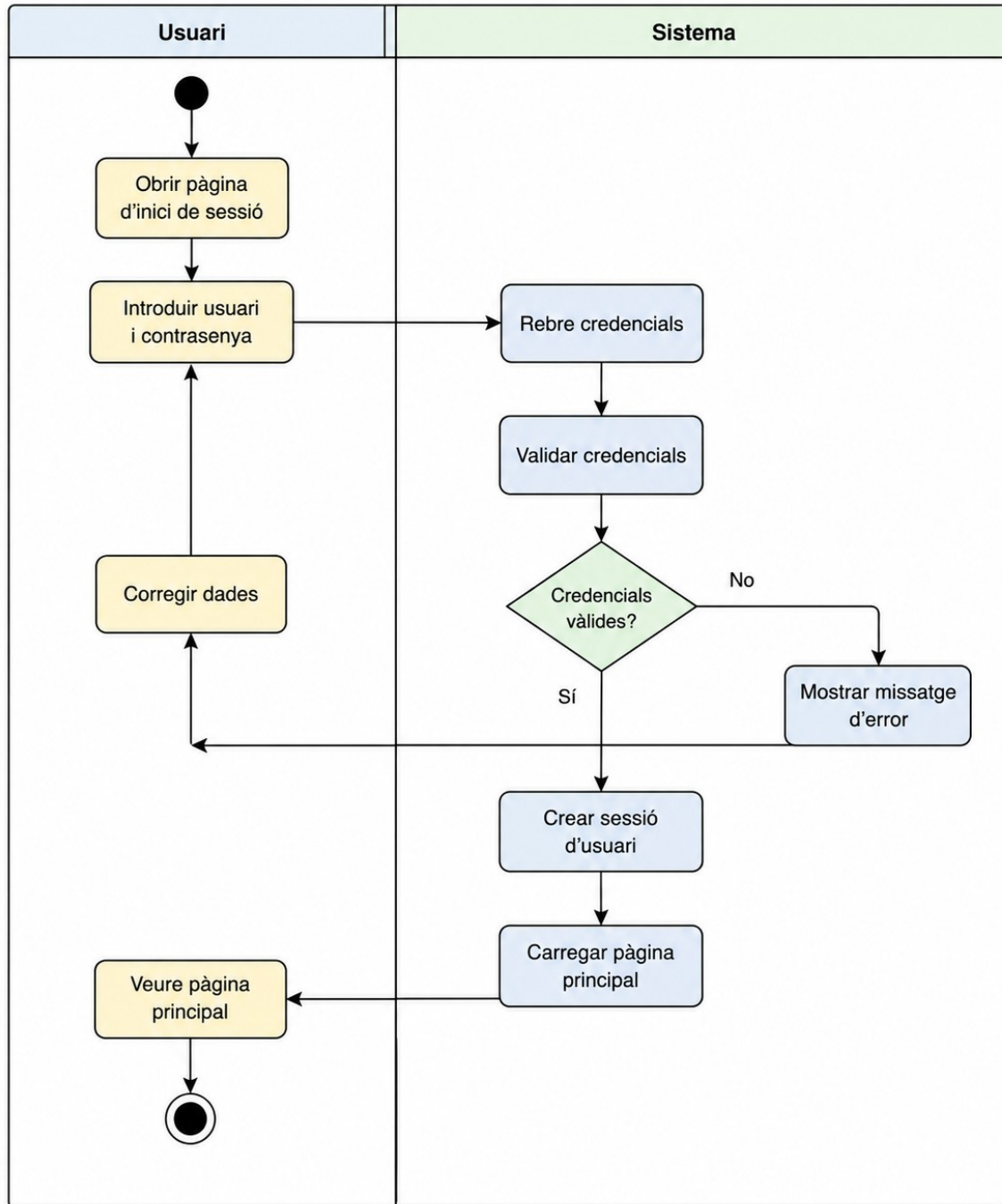
5. Diagrama d'Activitat

Mostra el flux de control d'un algorisme o procés de negoci pas a pas (molt similar a un organigrama o diagrama de flux tradicional).

- *Accions*: Rectangles amb cantonades arrodonides.
- *Decisions*: Rombes que ramifiquen el flux segons condicions ([si] / [no]).
- *Bifurcació i Unió (Fork/Join)*: Barres horitzontals o verticals gruixudes que permeten reflectir activitats que s'executen en paral·lel.

Exemple:

Diagrama d'Activitat - Procés d'Iniciar Sessió



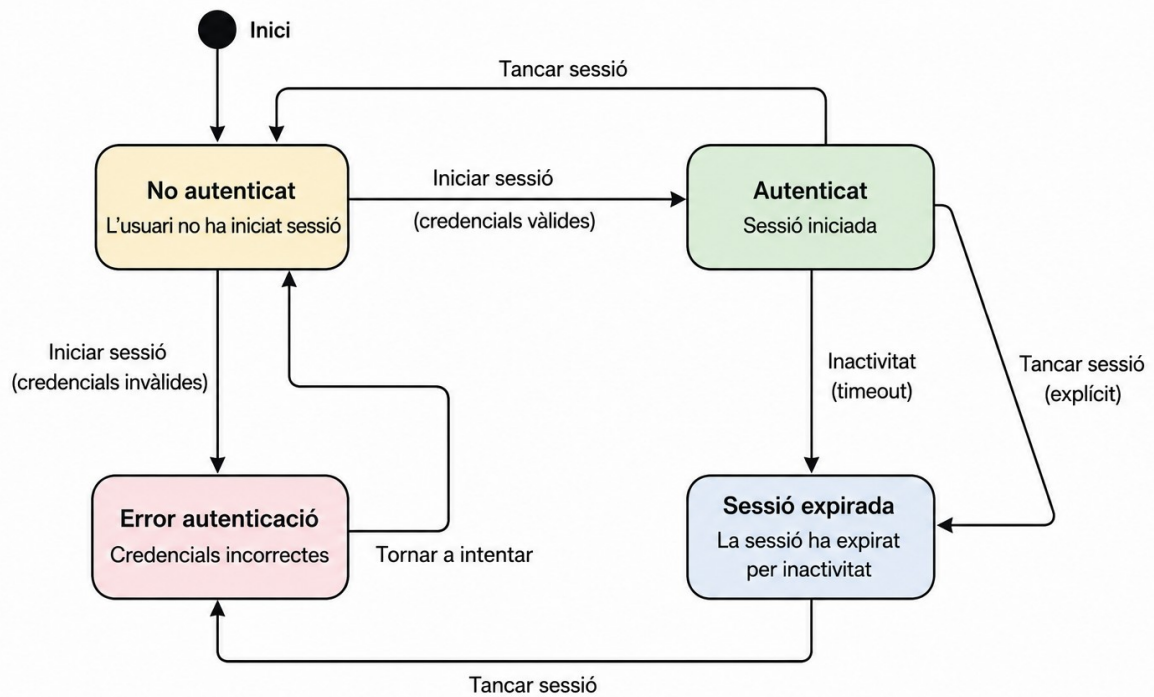
6. Diagrama d'Estats (Cicle de vida)

Mostra els **diferents estats pels quals passa un únic objecte** al llarg de la seua vida com a resposta a esdeveniments externs.

- *Estat*: Condició o situació de l'objecte en un moment donat (ex: Comanda *Pendent*, *Enviada*, *Lliurada*). Es representa amb rectangles arrodonits.
- *Transició*: Línia amb fletxa que unix un estat amb un altre, etiquetada amb l'esdeveniment que provoca el canvi.

Exemple:

Diagrama d'Estats – Sessió d'Usuari



--ooOOoo--